

Verificação e Validação

Profa. Rosilene Fernandes
f.rosilene@pucpr.br

Profa. Sofia T. T. M. Apuzzo
sofia.marshallowitz@pucpr.br

Prof. Marcelo A. G Bardi
marcelo.bardi@pucpr.br

Adaptações do material do prof. Marco Paludo e prof. Marcelo Walter

Agenda

Algumas ferramentas

Teste de Caixa Branca

Algumas ferramentas



Performace - JMeter

JMeter é uma aplicação gratuita, open source, multiplataforma e escrita em Java, sendo um projeto da Apache Software Foundation

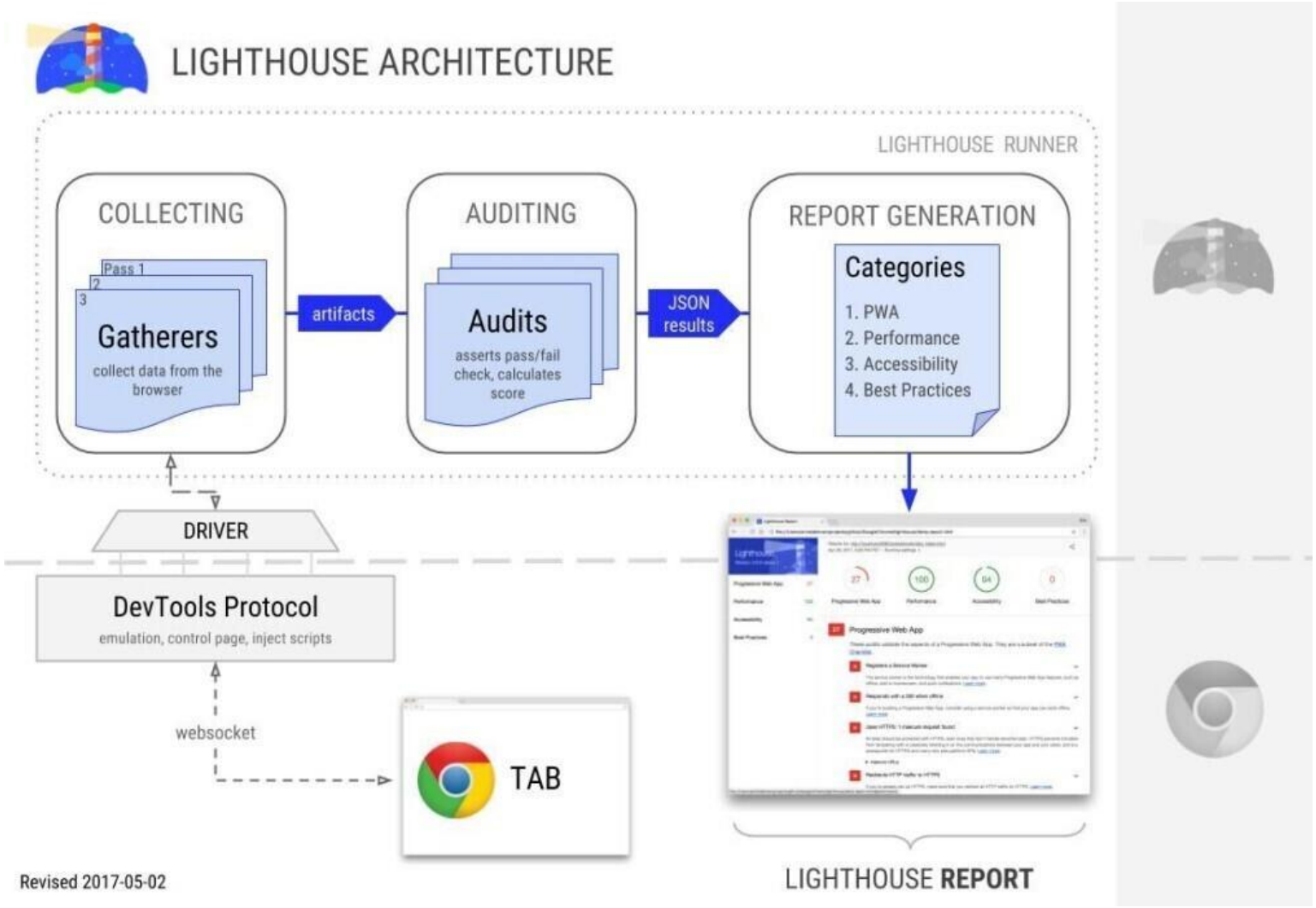
Desenvolvida inicialmente para testes em aplicações web, mas hoje é possível o utilizar para testes em recursos variados, como banco de dados, servidores de e-mail, requisições HTTP, FTP, TCP, SOAP e outros

Com o **JMeter é possível realizar os testes de performance** (carga e stress), tornando possível mensurar performance de uma aplicação web

https://jmeter.apache.org/



Desempenho, acessibilidade e otimização - Google Lighthouse



Desempenho, acessibilidade e otimização - Google Lighthouse

O relatório fornece análise abrangente de alguns itens, tais como:

- **Desempenho:** quão rápido está o desempenho do site (por exemplo, com que rapidez as páginas estão sendo carregadas, etc.)?
- **Progressive Web Application:** o site fornece uma experiência moderna e semelhante a um aplicativo para os visitantes?
- **Acessibilidade:** quão acessíveis são as páginas da Web? Quais são as vulnerabilidades?
- **Melhores práticas:** o site está de acordo com as boas práticas recomendadas?

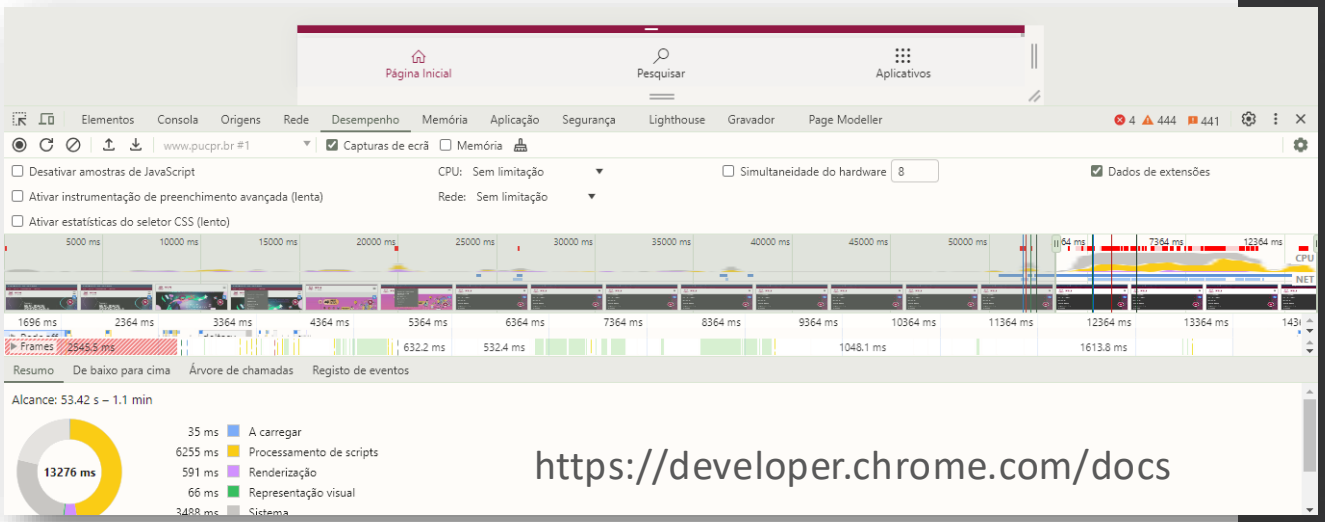
https://chrome.google.com/webstore/detail/lighthouse



Medir desempenho e simular diferentes tipos de dispositivos e navegadores - DevTools

Ajuda a analisar o desempenho e a eficiência das interações do usuário:

- Abra o **Chrome DevTools** (pressione F12 ou clique com o botão direito na página e selecione "Inspecionar")
- Navegue até a aba "**Performance**" e inicie a gravação enquanto interage com o sistema. Isso permite ver **onde o tempo é gasto e identificar possíveis problemas de usabilidade**



DevTools

https://developer.chrome.com/docs

Segurança, identificação de vulnerabilidades – OWASP Zap

Ferramenta de **teste de segurança** de aplicativos da web de código aberto projetada para ajudar desenvolvedores, testadores de segurança a identificar vulnerabilidades, permite:

- Escaneamento Passivo e Ativo: **detecta vulnerabilidades** de segurança no sistema
- Interceptação de Requisições HTTP: permite analisar e modificar **tráfego entre o navegador e a aplicação**
- Suporte a APIs REST e SOAP: verifica a **segurança de APIs**

https://www.zaproxy.org/



OWASP
Zed Attack Proxy

https://owasp.org/

Segurança, identificação de falhas de segurança e vulnerabilidades - Horusec

O **Horusec** é uma ferramenta open source que orquestra outras ferramentas de segurança e **identifica falhas de segurança ou vulnerabilidades em projetos**, centralizando todos os resultados em um banco de dados para análise e geração de métricas



https://horusec.io/site/

Horusec

Captura de imagens

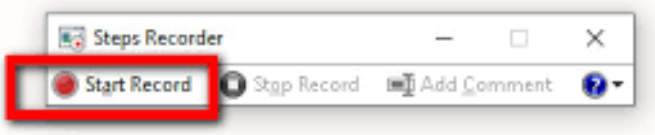


- Snipping Tool e Snip & Sketch (Windows): permite capturar a tela inteira, uma área específica, ou uma janela e inclui ferramentas básicas de anotação
  - **Snipping Tool:** Procure "Snipping Tool" no menu Iniciar
  - **Snip & Sketch:** Pressione Windows + Shift + S para abrir a ferramenta de captura



- Nimbus Screenshot & Screen Video Recorder (Extensão do Chrome): permite **capturar a tela e gravar vídeos**
  - Instale a extensão do Nimbus e utilize o ícone da extensão para capturar e editar imagens.

  - O **Gravador de Passos do Windows:** também conhecido como "Problem Steps Recorder" (PSR), é uma ferramenta que **captura capturas de tela e anotações a cada clique do mouse**, registrando uma sequência de ações na tela.





PUCPR

GRUPO MARISTA

Contato:

f.rosilene@pucpr.br
marcelo.bardi@pucpr.br
sofia.marshallowitz@pucpr.br



Verificação e Validação

Profa. Rosilene Fernandes
f.rosilene@pucpr.br

Profa. Sofia T. T. M. Apuzzo
sofia.marshallowitz@pucpr.br

Prof. Marcelo A. G Bardi
marcelo.bardi@pucpr.br

Adaptações do material do prof. Marco Paludo e prof. Marcelo Walter

Teste de Caixa Branca



Teste de Caixa Branca

- **Objetivo:** validar a estrutura interna e o funcionamento do código de software.

  Para conduzir um Teste da Caixa Branca, o conhecimento da linguagem de programação subjacente é essencial

  Exemplos:

  Teste unitário, teste de mutação ...

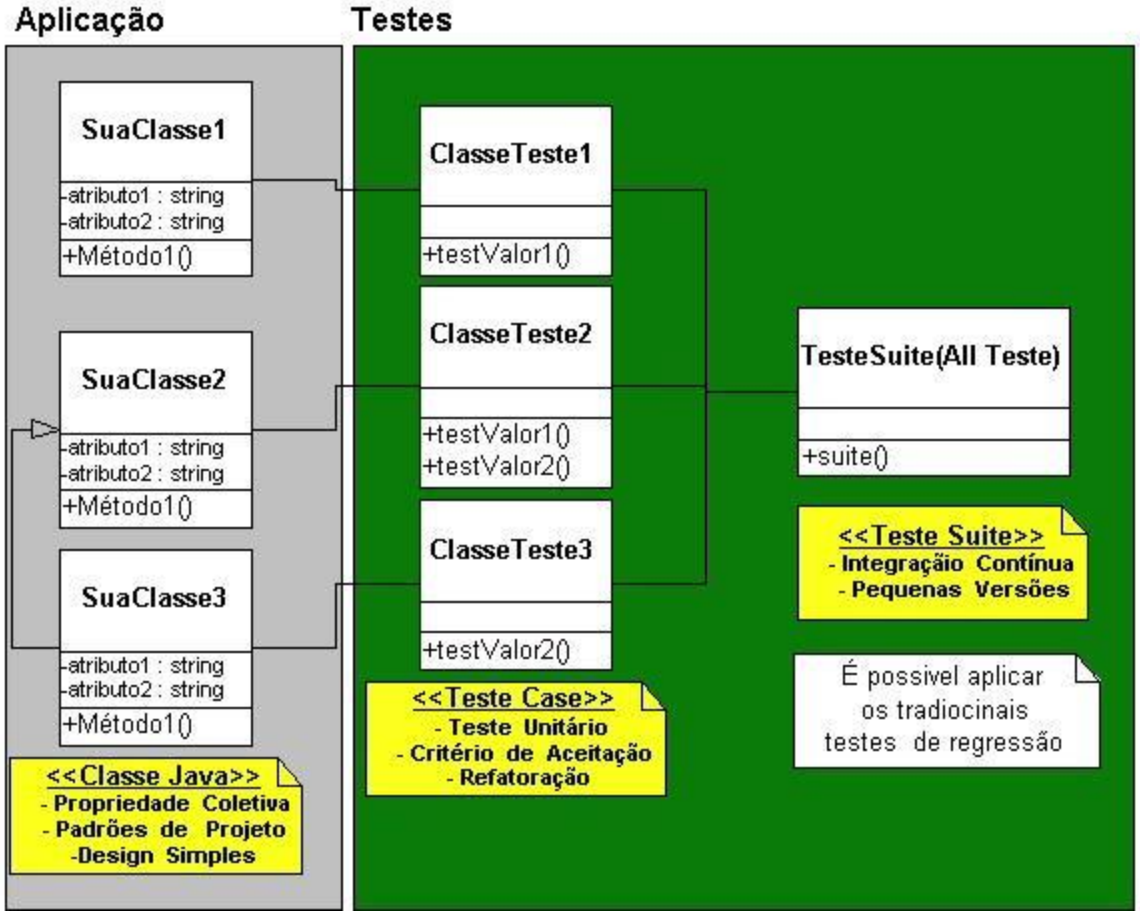
  Ferramentas:

  JUnit (Java), PHPUnit (PHP), Nunit (C#), Jest (JavaScript) , UnitTest (Python) ...

Teste Unitário: o que testar?

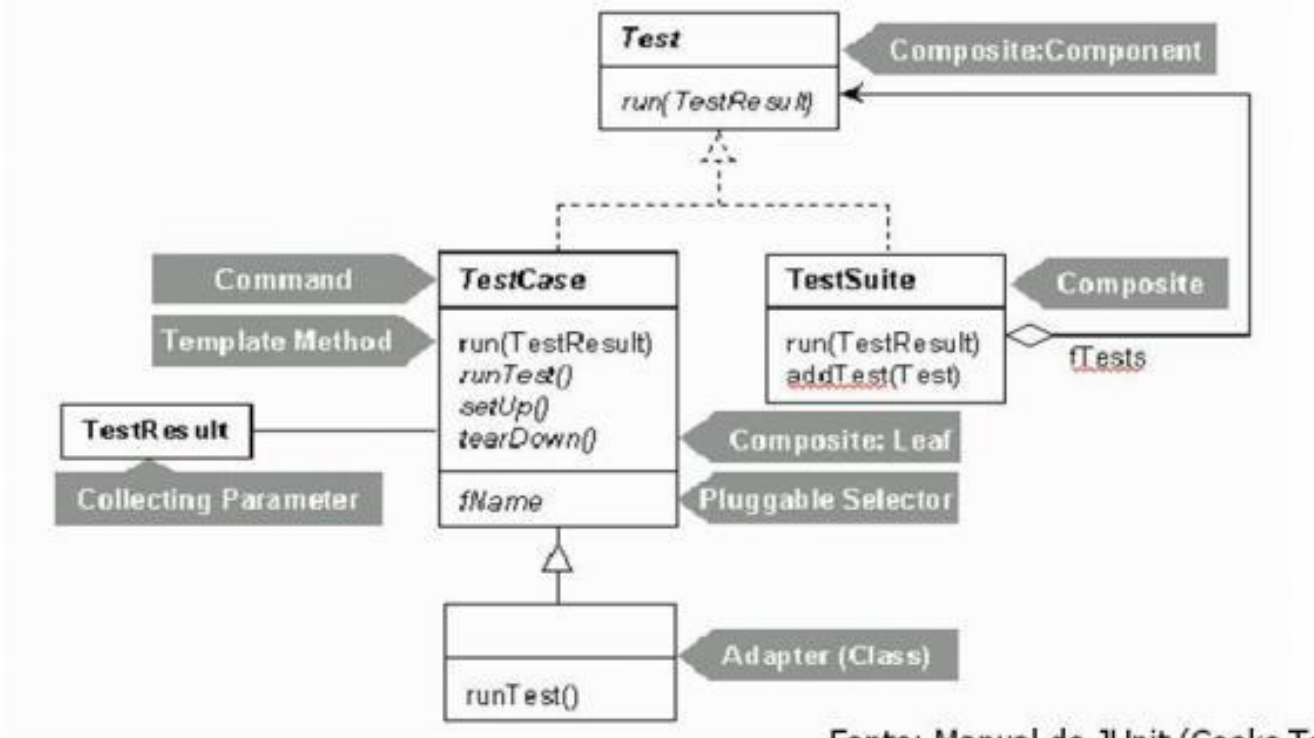
- A principal regra para saber o que testar é: “Tenha criatividade para imaginar as possibilidades de testes”
- Comece pelas mais simples e deixe os testes “complexos“ para o final
- Use apenas dados suficientes (não teste 10 condições se três forem suficientes)
- Não teste métodos triviais, tipo get e set
- No caso de um método set, só faça o teste caso haja validação de dados
- Achou um bug? Não conserte sem antes escrever um teste que o pegue (se você não o fizer, ele volta)

Teste Unitário: organização



Test Case (para cada classe)
Test Suite (executa vários test cases)

Teste Unitário: organização



Fonte: Manual do JUnit (Cooks Tour)

Test Case (para cada classe)
Test Suite (executa vários test cases)

Teste Unitário: exemplo

Imagine que estamos construindo uma **calculadora** e precisamos garantir a estrutura interna e o funcionamento da **função soma**:

```
1   function soma(a, b){
2     return a + b;
3   }
```

1



função que soma dois números e retorna o valor da soma

```
1   var resultado = soma(1, 2);
2   expect(resultado).to.equal(3);
```

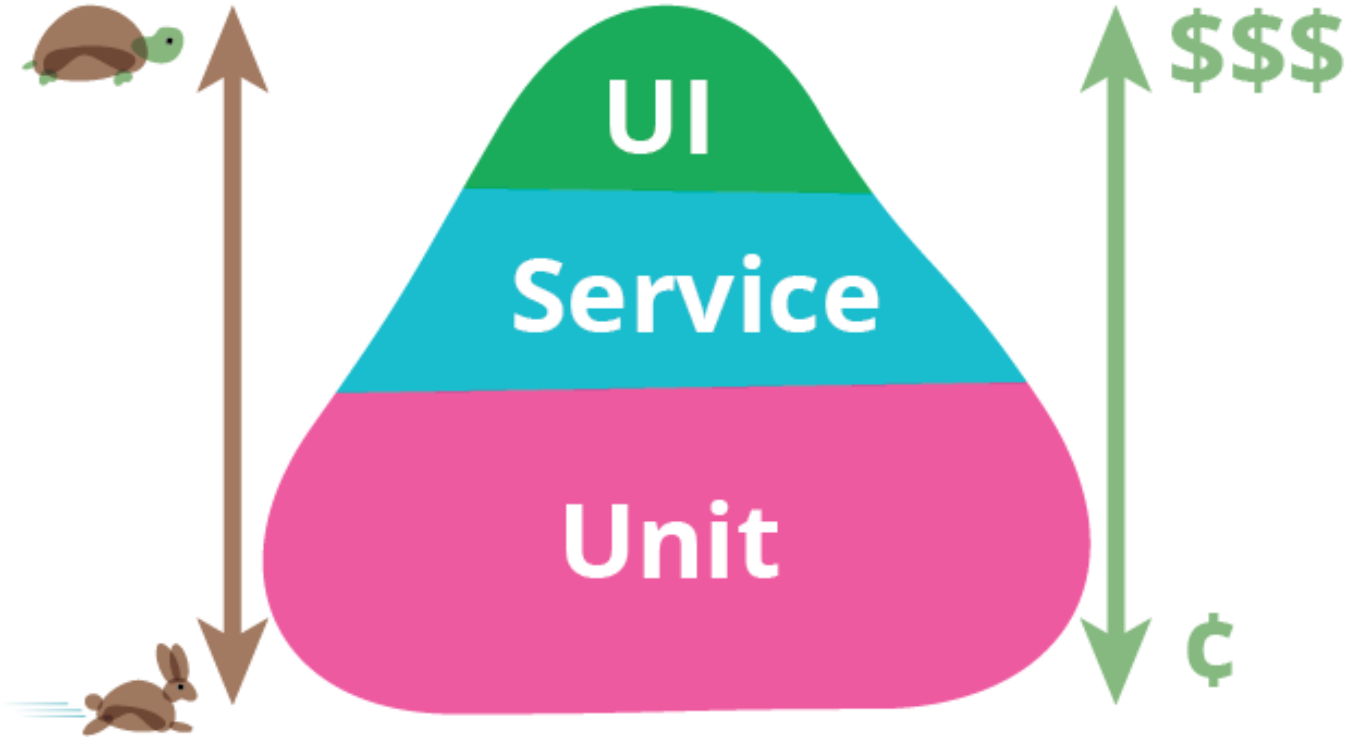
2



para testar este código, basta executar a função e verificar se seu valor de saída é o que esperamos

Pronto! Já temos um teste unitário

Teste Unitário



"Os **testes unitários são mais simples, rápidos e baratos** em relação aos testes a nível de usuário" **Mike Cohn**

Teste de Mutação

É possível **avaliar o conjunto de teste** com a inserção de pequenos defeitos no código-fonte da aplicação. A versão com defeitos inseridos é chamada de mutante (DeMillo et al. 1978) e é conhecida como Análise de Mutantes.

Funcionamento: **bugs (mutantes), são inseridos no código e os testes são executados em cima do código mutado**. Se pelo menos 1 dos testes quebrar ou tiver timeout, o mutante é considerado eliminado e aquele trecho de código alterado é considerado como coberto pelos testes.

Teste de Mutação: exemplo

Imagine que agora vamos verificar se os testes unitários construídos para a **função soma da calculadora** estão adequados:

```
1   function soma(a, b){
2     return a / b;
3   }
```

1



alterado o operador da função soma

```
1   var resultado = soma(1, 2);
2   expect(resultado).to.equal(3);
```

2



para testar este código, basta executar a função e verificar se seu valor de saída é diferente do que esperamos

Pronto! Já temos um teste de mutação

Teste de Mutação

Quais **outros mutantes** você consegue imagina para a **função soma**?

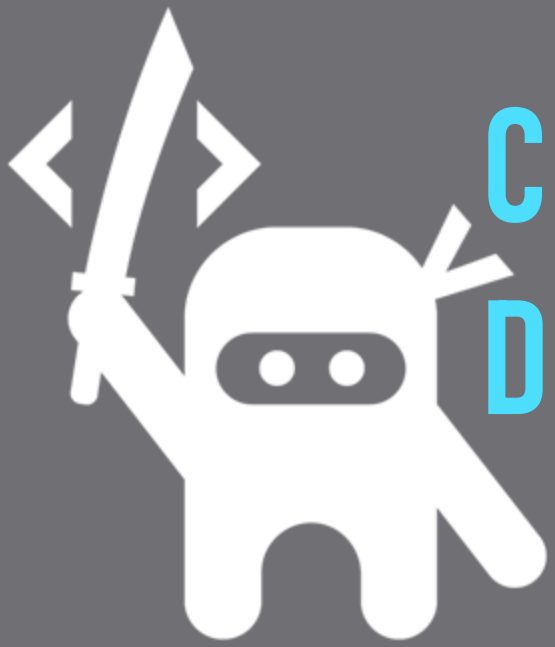
```
1   function soma(a, b){
2     return a + b;
3   }
```











Coding Dojo

Formativa - teste unitário

Desafio em

- **cinco** rodadas com
- **cinco** integrantes, implementar
- **cinco** funcionalidade de uma calculadora e
- **cinco** testes unitários

JAVA

```
public class Calculadora {

    public static int soma(int x, int y) {
        return x+ y;
    }
    public static void main(String[] args) {
        int resultado = Calculadora.soma(5, 3);
        System.out.println("A soma é: " + resultado);
    }
}
```

JUnit

```
import org.junit.Assert;
import org.junit.Test;

public class CalculadoraTest {
    @Test
    public void testSoma() {
        int resultado = Calculadora.soma(5, 3);
        Assert.assertEquals(8, resultado);
    }
}
```

Python

```
class Calculadora:
        def soma(self, num1, num2):
                return num1 + num2

calc = Calculadora()
resultado = calc.soma(5, 3)
print(resultado)
```

UnitTest

```
import unittest
class TestCalculadora(unittest.TestCase):
        def test_soma(self):
                calc = Calculadora()

# Teste com números positivos
resultado = calc.soma(5, 3)
self.assertEqual(resultado, 8)

if _name_ == '_main_':    unittest.main()
```

Php

```php
<?php
	class Calculadora {
		public function soma($num1, $num2) {
			return $num1 + $num2;
		}
	}
	$calc = new Calculadora();
	$resultado = $calc->soma(5, 3);
	echo $resultado;
?>
```

```php
use PHPUnit\Framework\TestCase;

require 'Calculadora.php'; // incluir o arquivo da classe Calculadora

class CalculadoraTest extends TestCase {
	public function testSoma() {
		$calc = new Calculadora();

	// Teste com números positivos
	$resultado = $calc->soma(5, 3);
	$this->assertEquals(8, $resultado);
	}
}
```

Coding Dojo

5 funcionalidade

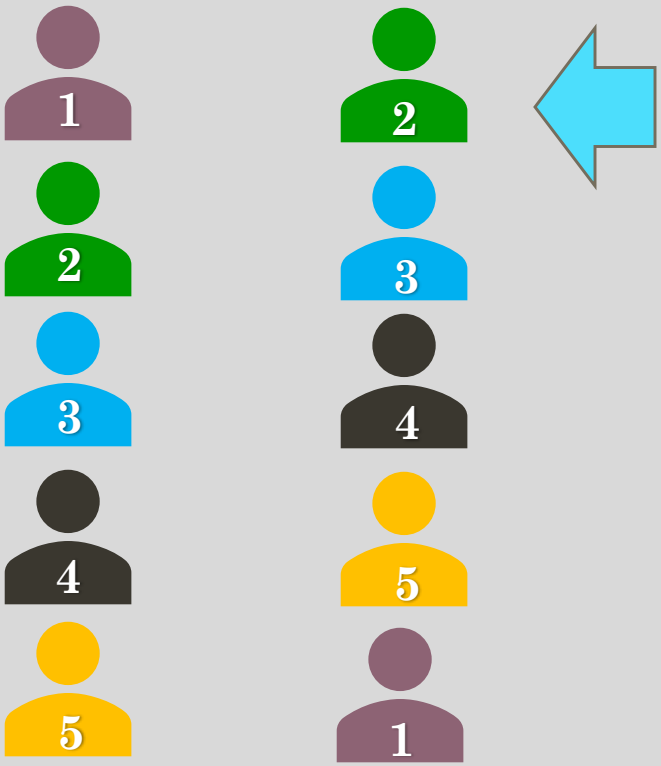
5 teste unitário





Piloto

Copiloto



- O fluxo é **composto de ciclos de 3 min minutos**
- O **piloto é o único que programa** e não deve ser interrompido pela plateia, onde realiza programação pareada com o **copiloto**, que é a **única pessoa que tem permissão para falar**, ajudar e dar opinião
- A plateia assiste e não fala até o final de cada ciclo
- No intervalo de cada ciclo a **dupla explica sua linha de pensamento** e o que está fazendo, o **piloto vai para a plateia, o copiloto assume a posição de piloto** e um voluntário (de preferência quem ainda não programou) assume o copiloto

Coding Dojo

5 funcionalidade

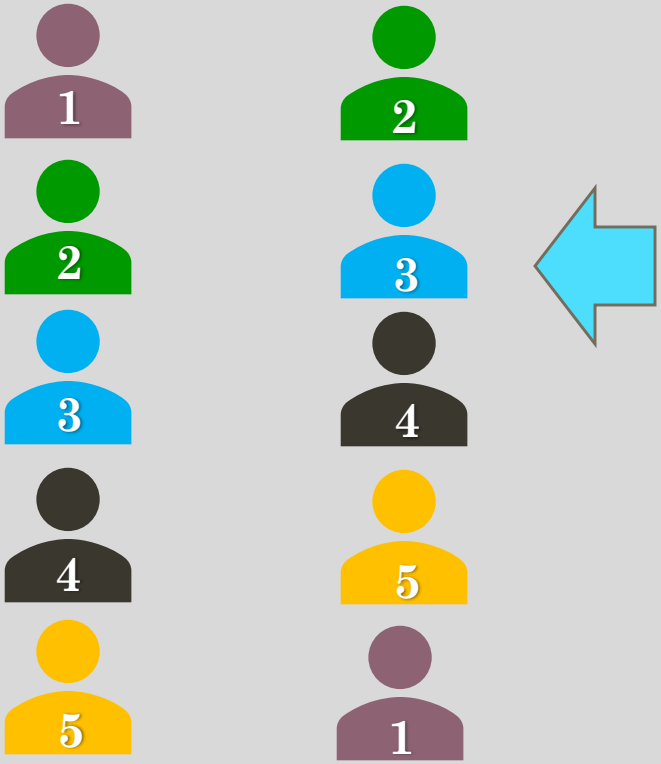
5 teste unitário





Piloto

Copiloto



- O fluxo é **composto de ciclos de 3 min minutos**
- O **piloto é o único que programa** e não deve ser interrompido pela plateia, onde realiza programação pareada com o **copiloto**, que é a **única pessoa que tem permissão para falar**, ajudar e dar opinião
- A plateia assiste e não fala até o final de cada ciclo
- No intervalo de cada ciclo a **dupla explica sua linha de pensamento** e o que está fazendo, o **piloto vai para a plateia, o copiloto assume a posição de piloto** e um voluntário (de preferência quem ainda não programou) assume o copiloto

Coding Dojo

5 funcionalidade

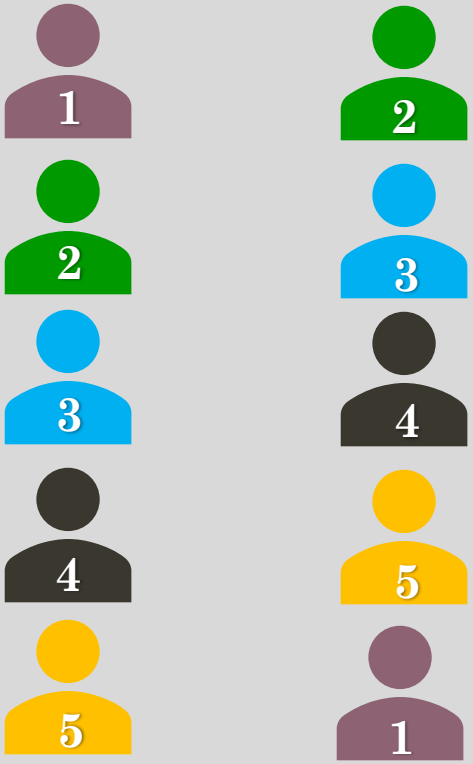
5 teste unitário





Piloto

Copiloto



- O fluxo é **composto de ciclos de 3 min minutos**
- O **piloto é o único que programa** e não deve ser interrompido pela plateia, onde realiza programação pareada com o **copiloto**, que é a **única pessoa que tem permissão para falar**, ajudar e dar opinião
- A plateia assiste e não fala até o final de cada ciclo
- No intervalo de cada ciclo a **dupla explica sua linha de pensamento** e o que está fazendo, o **piloto vai para a plateia, o copiloto assume a posição de piloto** e um voluntário (de preferência quem ainda não programou) assume o copiloto

Coding Dojo

5 funcionalidade

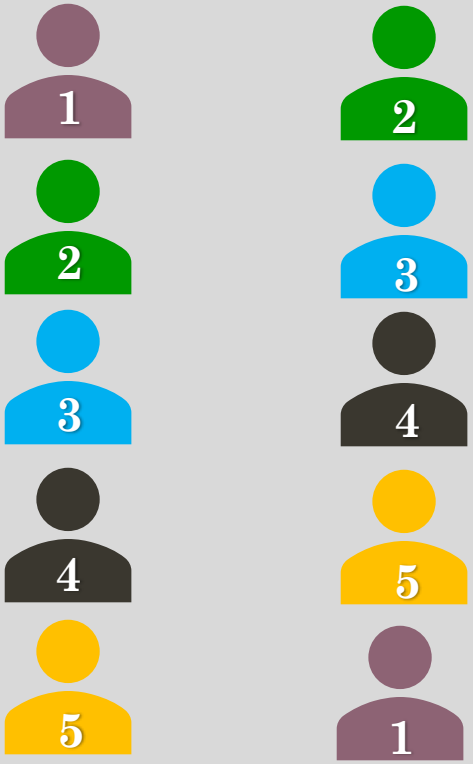
5 teste unitário





Piloto

Copiloto



- O fluxo é **composto de ciclos de 3 min minutos**
- O **piloto é o único que programa** e não deve ser interrompido pela plateia, onde realiza programação pareada com o **copiloto**, que é a **única pessoa que tem permissão para falar**, ajudar e dar opinião
- A plateia assiste e não fala até o final de cada ciclo
- No intervalo de cada ciclo a **dupla explica sua linha de pensamento** e o que está fazendo, o **piloto vai para a plateia, o copiloto assume a posição de piloto** e um voluntário (de preferência quem ainda não programou) assume o copiloto

Coding Dojo

5 funcionalidade

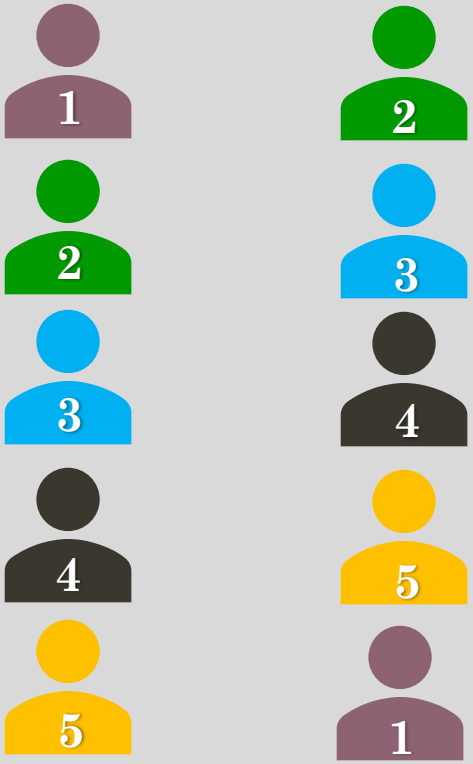
5 teste unitário



Piloto



Copiloto





- O fluxo é **composto de ciclos de 3 min minutos**
- O **piloto é o único que programa** e não deve ser interrompido pela plateia, onde realiza programação pareada com o **copiloto**, que é a **única pessoa que tem permissão para falar**, ajudar e dar opinião
- A plateia assiste e não fala até o final de cada ciclo
- No intervalo de cada ciclo a **dupla explica sua linha de pensamento** e o que está fazendo, o **piloto vai para a plateia, o copiloto assume a posição de piloto** e um voluntário (de preferência quem ainda não programou) assume o copiloto

Projeto RA2:

Escolha um sistema que você tenha acesso a especificação e ao código fonte (ex.: sistema desenvolvido na disciplina Experiência Criativa ou em outra disciplina), para elaborar e executar um plano de teste

tipos de testes variados
mínimo oito casos de teste por integrante

Equipe: até 6 alunos

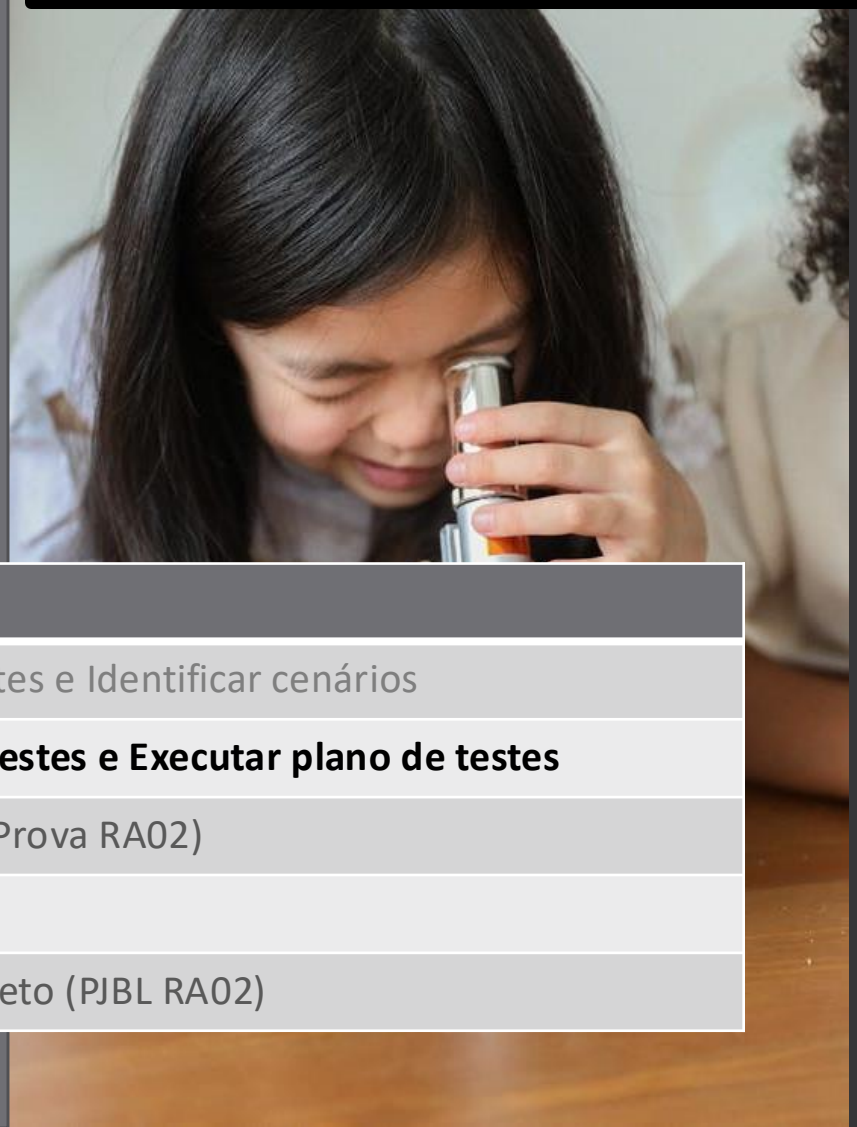
Sugestão: utilize os *templates* fornecidos

Agenda:

Data	Atividades
09 e 11/09	Elaborar plano de testes e Identificar cenários
16 e 18/09	**Especificar casos de testes e Executar plano de testes**
23/09	Avaliação Individual (Prova RA02)
30/09	Semana Acadêmica
02/10	Apresentação do projeto (PJBL RA02)

18/09→

- **Especificação e execução de pelo menos 50% dos casos de testes estimados**





PUCPR

GRUPO MARISTA

Contato:

f.rosilene@pucpr.br
marcelo.bardi@pucpr.br
sofia.marshallowitz@pucpr.br